

PDU Relay

Installation guide – deploying the relay service and web interface that let the PDU Ctrl.grid DashBoard panel power-cycle USP-PDU-Pro outlets.

What this is

The DashBoard panel (**PDU Ctrl.grid**) cannot log into the UniFi Network controller directly – its scripting engine can only send plain HTTP requests with no custom headers, so it can't carry the cookie/CSRF session UniFi's outlet-control API requires. This relay does that login on the panel's behalf and exposes the endpoints the panel calls instead: a PDU list, an outlet list per PDU, and a power-cycle action per outlet.

It also serves a small **web interface** (open `http://<relay-host>:8090/` in a browser) for setup and day-to-day use, so nothing has to be configured by editing files over SSH: a one-time login page creates an admin account, a Connection Settings page holds the UniFi controller host/credentials, and a PDUs page shows every discovered PDU with live outlet status and reboot buttons – the same control the touch panel has, from any browser on the network.

The web UI's login only protects that configuration/browser surface. The three endpoints the touch panel calls (`/pdu/list` and friends) have no authentication of their own – they can't, since the panel's scripting engine can't send an auth header – so they stay reachable to anything that can reach the port. Restrict that at the firewall (Step 6), not with a login.

Prerequisites

- A Debian (or Debian-based) LXC container or VM reachable from both the UniFi controller and the Ultritouch touch panel (and, ideally, whoever will administer it).
- A local UniFi admin account (username + password) – not a Ubiquiti cloud SSO login. Outlet control only exists on the controller's legacy REST API, which needs a real logged-in session.
- The USP-PDU-Pro already adopted in the UniFi Network application.

Step 1 – Install Node.js

The relay needs Node 18 or newer (it uses the built-in `fetch`). Debian's stock `apt` package is often older, so install from NodeSource:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version
```

Step 2 – Copy the relay onto the container

From wherever this project's files currently live, copy the whole PDU Relay folder onto the container (replace /path/to/PDU Relay and user@debian-host with your actual source path and SSH target):

```
scp -r "/path/to/PDU Relay" user@debian-host:/opt/pdu-relay
```

Step 3 – Install dependencies and start the relay

SSH into the container, install dependencies, and start the relay in the foreground for the first run:

```
cd /opt/pdu-relay
npm install
node pdu-relay-server.js
```

You should see PDU relay listening on :8090. Nothing needs to be edited by hand here – there's no .env to fill in for the UniFi connection. A config.json file is created automatically the first time settings are saved through the web UI in the next step (permissions restricted to the owner, since it holds the UniFi password).

Step 4 – Finish setup in the browser

From a browser on the same network, open:

<http://debian-host:8090/>

4a. Create the admin account – this is the login for this web page only, separate from UniFi. Pick a username and an 8+ character password.

4b. Open Connection Settings and fill in:

UDM Host / IP – your UDM SE's IP, e.g. 192.168.1.1

Local admin username / password – a local UniFi admin account

Leave site name, self-signed certificate, and PDU model at their defaults unless you have a specific reason to change them, then press **Save**.

4c. Open the PDUs tab. Within a few seconds it should list every discovered PDU with live outlet buttons – green for on, red for off. Clicking one power-cycles that outlet.

If the PDUs tab shows an error instead, it will say why (bad credentials, unreachable host, connection timeout, etc.) – fix that before moving on to the touch panel.

Stop the foreground process (Ctrl-C) once this works – Step 5 sets it up to run permanently.

Step 5 – Run it as a persistent service

So the relay survives reboots and doesn't die when an SSH session ends, run it under systemd. Create /etc/systemd/system/pdu-relay.service:

```
[Unit]
Description=PDU Relay for DashBoard PDU Ctrl.grid
```

```
After=network-online.target
Wants=network-online.target
```

```
[Service]
Type=simple
WorkingDirectory=/opt/pdu-relay
ExecStart=/usr/bin/node /opt/pdu-relay/pdu-relay-server.js
Restart=on-failure
RestartSec=5
User=nobody
Environment=NODE_ENV=production
```

```
[Install]
WantedBy=multi-user.target
```

Swap `User=nobody` for a dedicated account if preferred – whichever user runs it just needs read/write access to `/opt/pdu-relay/config.json`. Then enable it:

```
sudo chown -R nobody:nogroup /opt/pdu-relay
sudo systemctl daemon-reload
sudo systemctl enable --now pdu-relay
sudo systemctl status pdu-relay
```

The admin account and connection settings from Step 4 are already saved in `config.json` and will still be there. Check logs any time with:

```
sudo journalctl -u pdu-relay -f
```

Step 6 – Firewall

The relay's port (default 8090) serves both the unauthenticated panel endpoints and the login-gated web UI. Restrict access to only the devices that need it: the Ultritouch panel's IP (for outlet control) and whichever admin machines will manage it (for the web UI) – not the whole LAN.

Step 7 – Point the panel at the relay

Open **PDU Ctrl.grid** on the Ultritouch. It will show only the **Settings** tab until a PDU is found. Enter:

Relay Host – the Debian container's IP or hostname

Relay Port – 8090 (or whatever port the relay is listening on)

Then press **Connect / Refresh PDUs**. A tab should appear for each PDU, matching what the web UI's PDUs page already showed in Step 4.

Using the web UI afterward

The web UI stays available for as long as the relay runs – there's no need to go back to the command line for routine use:

PDUs – live outlet status and reboot buttons, same as the touch panel.

Connection Settings – update the UniFi host/credentials any time; changes apply immediately, no restart needed.

Admin Account – change the web UI's own login username/password.

Troubleshooting

PDUs tab shows an error – the message is specific (bad credentials, connection timeout, "UniFi connection isn't configured yet", etc.). Fix it in Connection Settings and it re-checks automatically every few seconds.

No PDUs found, but the connection itself works – the PDU may not be adopted, or the PDU model code on the Connection Settings page may not match this firmware's reported model.

Forgot the web UI's admin password – stop the service, delete `config.json`, and restart it. This clears the admin login *and* the saved UniFi connection settings, so Step 4 will need to be repeated.

Touch panel can't reach the relay – check the firewall rule from Step 6 actually allows the panel's specific IP, and that the relay service is running (`systemctl status pdu-relay`).